

Understanding Trajectron++: A Per-Trajectory Performance Analysis

Simon Drauz and Zoe Matrullo
May 16, 2026

Showcase
Figure

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements

[figure placeholder]

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**

[figure placeholder]

- **Task:** forecast future (x, y) positions of agents (pedestrians, cyclists, vehicles) from observed past movements
- Challenging due to **complex agent interactions** and influence of **static context**
- Essential for **safe path planning** and **collision avoidance** in autonomous vehicles

[figure placeholder]

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*
 - ▶ Outputs a distribution over future trajectories

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- State-of-the-art probabilistic trajectory prediction model [Salzmann et al., 2020]
- *Spatio-temporal graph encoder*
 - ▶ LSTM per agent
 - ▶ edges encode agent-agent interactions
- *CVAE decoder*
 - ▶ Outputs a distribution over future trajectories
- Handles *heterogeneous* agent types jointly (pedestrian, bicycle, car) and integrates HD map context

trajdata [NVLabs]

- Unified interface to many trajectory datasets (nuScenes, ETH/UCY, ...)
- ⇒ Our analysis pipeline applies to *any* trajdata-supported dataset

- 1 000 urban driving scenes, Boston & Singapore

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 scenes (train / val / test)

[figure placeholder]

- 1 000 urban driving scenes, Boston & Singapore
- Agent types: pedestrians, cyclists, vehicles
- Rich sensor suite: camera, LiDAR, radar, GPS, HD map
- Standard split: 700 / 150 / 150 scenes (train / val / test)

[figure placeholder]

Prediction metrics:

ml_ade	most-likely ADE ← used
ml_fde	most-likely FDE
min_ade_5	best-of-5 ADE
nll_mean	negative log-likelihood

- Some trajectories are inherently hard, others trivially easy—but *why*?

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

- ① Which trajectory and scene characteristics affect model performance?

⇒ **Three research questions:**

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- ① Which trajectory and scene characteristics affect model performance?
- ② What are the *success and failure modes* that explain model performance?

- Some trajectories are inherently hard, others trivially easy—but *why*?
- A single average ADE/FDE gives no insight into reasons for failure and success
- No understanding of which trajectory or scene characteristics drive per-trajectory prediction error

⇒ **Three research questions:**

- ① Which trajectory and scene characteristics affect model performance?
- ② What are the *success and failure modes* that explain model performance?
- ③ How do model settings (history window, prediction horizon, attention radius) affect performance?

Outline

- 1 Motivation
- 2 Analysis Pipeline**
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

Track 1 — Full Dataset

- Train Trajectron++ → per-trajectory ADE
- join trajectory + scene characteristics
- interpretable models → feature effects
- performance regime clustering
- cluster inspection

Clustering + cluster inspection run only on the full-dataset path.

Track 2 — Mini Dataset (Settings Sweep)

- Cartesian sweep over model settings
- combine runs
- interpretable models
- feature effects with settings as predictors
(trajectory/scene chars as controls)

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to
an individual trajectory

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Standard Trajectron++

- Evaluation outputs *batch-averaged*
ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics* by modifying Trajectron++'s evaluation loop

Standard Trajectron++

- Evaluation outputs *batch-averaged* ADE / FDE
- No way to link prediction error back to an individual trajectory
- Cannot study what makes a trajectory hard or easy

Our Modification

- Recover *individual trajectory loss & characteristics* by modifying Trajectron++'s evaluation loop
- `data_idx`: stable trajdata identifier, used to join evaluation output with computed trajectory & scene characteristics

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation**
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

Agent motion features

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

Agent motion features

- Speed: mean, max, std
- Acceleration: mean, max
- Jerk: mean, max
- Heading change per second
- Min. neighbour distance
- Collision indicator

Scene context features

- Number of agents in scene
- Number of vehicles
- Scene bounding box area
- Spatial agent density

⇒ **18 candidate features** joined with
Trajectron++ evaluation output via `data_idx`

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential
- Log transformation ($\log_{10}$) applied to stabilise variance and improve model fit

[figure placeholder: before/after distribution]

- `ml_ade` is strongly *right-skewed* (skewness = 2.59)
- Small errors are very frequent; large errors are rare but influential
- Log transformation ($\log_{10}$) applied to stabilise variance and improve model fit
- Models are trained on log-scale target, evaluated on original scale

[figure placeholder: before/after distribution]

- Features ranked by Mutual Information (MI) with target

[figure placeholder: MI elbow plot]

- Features ranked by Mutual Information (MI) with target
- MI captures *non-linear* dependencies, unlike correlation

[figure placeholder: MI elbow plot]

- Features ranked by Mutual Information (MI) with target
- MI captures *non-linear* dependencies, unlike correlation
- Elbow detection: largest single-step drop in ranked MI curve separates informative features from low-signal tail

[figure placeholder: MI elbow plot]

- Features ranked by Mutual Information (MI) with target
- MI captures *non-linear* dependencies, unlike correlation
- Elbow detection: largest single-step drop in ranked MI curve separates informative features from low-signal tail
- Features below the elbow are dropped

[figure placeholder: MI elbow plot]

- Features ranked by Mutual Information (MI) with target
- MI captures *non-linear* dependencies, unlike correlation
- Elbow detection: largest single-step drop in ranked MI curve separates informative features from low-signal tail
- Features below the elbow are dropped
- **18 → 10 candidate features**

[figure placeholder: MI elbow plot]

- Collinear features split SHAP attribution arbitrarily and make GAM smooth terms unidentifiable

- Collinear features split SHAP attribution arbitrarily and make GAM smooth terms unidentifiable
- Iterative VIF elimination (threshold = 5)

- Collinear features split SHAP attribution arbitrarily and make GAM smooth terms unidentifiable
- Iterative VIF elimination (threshold = 5)
- At each step: drop the feature above threshold with the *lowest* $|\rho_{\text{Spearman}}|$ with target

Dropped (VIF > 5):

mean_speed	VIF = 357
max_jerk	VIF = 20
mean_jerk	VIF = 12
max_acceleration	VIF = 10
path_efficiency	VIF = 9

- Collinear features split SHAP attribution arbitrarily and make GAM smooth terms unidentifiable
- Iterative VIF elimination (threshold = 5)
- At each step: drop the feature above threshold with the *lowest* $|\rho_{\text{Spearman}}|$ with target
- Model settings always protected

Dropped (VIF > 5):

mean_speed	VIF = 357
max_jerk	VIF = 20
mean_jerk	VIF = 12
max_acceleration	VIF = 10
path_efficiency	VIF = 9

6 final features:

std_speed, mean_acceleration, min_neighbor_distance,
heading_change_per_sec, scene_num_VEHICLE, scene_density_VEHICLE

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling**
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

- We have per-trajectory `m1_ade` — but raw error values alone do not tell us *why* some trajectories are hard

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction
- **Interpretable models** let us directly read off feature effects: how does speed, heading change, or scene density affect prediction error?

- We have per-trajectory `ml_ade` — but raw error values alone do not tell us *why* some trajectories are hard
- A black-box model can predict ADE well but gives no insight into which features drive it or in which direction
- **Interpretable models** let us directly read off feature effects: how does speed, heading change, or scene density affect prediction error?
- This turns ADE into actionable understanding of model behaviour

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions

We tested two interpretable modeling approaches — either can be used for the analysis.

GAM (Generalized Additive Model)

- Models target as a sum of smooth functions: $f(x) = \sum_j s_j(x_j)$
- One spline per feature — no interaction terms
- p-values for significance testing
- Directly readable smooth effects

XGBoost

- Gradient-boosted decision trees
- Flexibly captures non-linearities and feature interactions
- Post-hoc interpretation via SHAP values

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`

XGBoost — SHAP Values

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
- Holding all other features at their mean

XGBoost — SHAP Values

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
- Holding all other features at their mean
- Confidence bands from the spline fit

XGBoost — SHAP Values

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
 - Holding all other features at their mean
 - Confidence bands from the spline fit
- ⇒ How does ADE change as, e.g., speed increases?

XGBoost — SHAP Values

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
 - Holding all other features at their mean
 - Confidence bands from the spline fit
- ⇒ How does ADE change as, e.g., speed increases?

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions

GAM — Partial Dependence

- The fitted smooth $\hat{s}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
 - Holding all other features at their mean
 - Confidence bands from the spline fit
- ⇒ How does ADE change as, e.g., speed increases?

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions
- *Local*: why was this trajectory's ADE high?

GAM — Partial Dependence

- The fitted smooth $\hat{\sigma}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
 - Holding all other features at their mean
 - Confidence bands from the spline fit
- ⇒ How does ADE change as, e.g., speed increases?

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions
- *Local*: why was this trajectory's ADE high?
- *Global*: average $|\text{SHAP}|$ gives overall feature importance and direction

GAM — Partial Dependence

- The fitted smooth $\hat{\sigma}_j(x_j)$ shows the marginal effect of feature j on `ml_ade`
 - Holding all other features at their mean
 - Confidence bands from the spline fit
- ⇒ How does ADE change as, e.g., speed increases?

XGBoost — SHAP Values

- Each prediction is decomposed into per-feature contributions
 - *Local*: why was this trajectory's ADE high?
 - *Global*: average $|\text{SHAP}|$ gives overall feature importance and direction
- ⇒ Same question answered from a different angle

- **Problem:** hyperparameter tuning risks overfitting to a particular data split
- **Solution:** nested cross-validation
 - ▶ Outer folds: evaluate generalization of the full tuning procedure
 - ▶ Inner folds: tune hyperparameters on each outer-fold training set
- OOF evaluation: inspect out-of-fold predictions for instability, outlier folds, bias
- **Conclusion:** tuning generalises stably → proceed to full-data tuning + refit

Final models are refit on *all* data with optimal hyperparameters — no held-out test set, because the goal is understanding, not unbiased prediction.

- Candidates: linear GAM, linear GAM on log-transformed target, Gamma GAM
- Selection criterion: lowest RMSE
- ...

Final Model Performance (OOF)

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance**
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions

XGBoost (global SHAP)

GAM (p-values of smooth effects)



Feature Effects — Trajectory Characteristics



Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes**
- 7 Model Settings Analysis
- 8 Conclusions

- ① Divide trajectories into difficulty groups based on `m1_ade` (easy / medium / hard)
- ② Run clustering on trajectory + scene characteristics per group
 - ▶ Option A: original feature space
 - ▶ Option B: UMAP-reduced space (dimensions chosen via trustworthiness score + elbow rule)
- ③ Assess clustering quality with DBCV score
- ④ Export identified clusters (performance regimes)



Regime Inspection — What Drives Error in Each Regime?

● ...

Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis**
- 8 Conclusions

- Dataset: nuScenes mini (full dataset too expensive for repeated runs)
- Cartesian sweep over:
 - ▶ `history_sec`
 - ▶ `prediction_sec`
 - ▶ `attention_radius_m`
- Pooled model: model setting variables + trajectory/scene features as control variables
- Feature normalization: setting-agnostic versions to avoid mechanical confounding (e.g. `mean_path_length_per_second` instead of `path_length`)



Outline

- 1 Motivation
- 2 Analysis Pipeline
- 3 Data Preparation
- 4 Interpretable Modeling
- 5 Feature Effects & Importance
- 6 Performance Regimes
- 7 Model Settings Analysis
- 8 Conclusions**

- RQ1 — Characteristics driving performance: ...
- RQ2 — Performance regimes: ...
- RQ3 — Model settings: ...

● ...